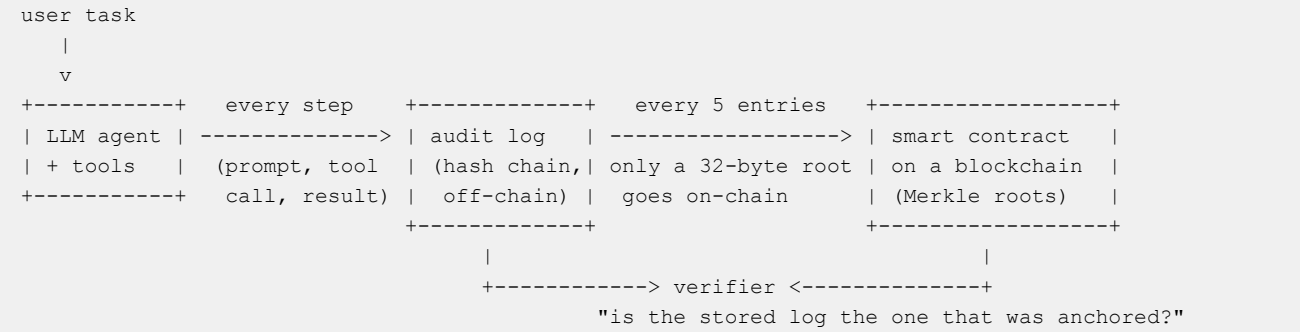
Technologies Explained

Project: Auditable AI Agents: An Immutable On-Chain Audit Trail for LLM Agent Actions
Author: Manikandan Sankaranarayanan
Audience: written for someone who has not used these technologies before. Every section says what the thing is, why this project uses it, where it lives in the code, and how to check that it works.

---

0. The whole idea in one picture



An AI agent does work and every step is written to a log. The log is protected two ways: each entry carries the fingerprint of the previous entry (a hash chain), and every few entries a single fingerprint of the whole batch (a Merkle root) is written to a blockchain. If anyone later edits, removes, inserts or reorders log entries, the verifier notices, and it can say where.

The rest of this document explains each building block, from the bottom up.

---

1. Hashing and SHA-256

What it is. A hash function turns any data into a short fixed-size fingerprint. SHA-256 always produces 32 bytes (64 hexadecimal characters). Properties that matter here:

- The same input always gives the same output.
- Changing one character of the input changes the output completely and unpredictably.
- You cannot work backwards from the fingerprint to the input, and in practice you cannot find two different inputs with the same fingerprint.

Example from the test suite: the SHA-256 of the three letters `abc` is `ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad`.

Why this project uses it. A fingerprint is a compact, unforgeable summary of a log entry. If the fingerprint is protected, the entry is protected.

Why SHA-256 specifically. It is built into Python (`hashlib`) and into Solidity (`sha256()`), so the Python verifier and the smart contract compute identical values. (Ethereum's own default, Keccak-256, is not the

same function as Python's `sha3_256`, which would have caused mismatches.)

Where. `src/core/hashing.py` (`sha256_hex`). **Test.** `tests/test_audit.py::test_sha256_known_vector`.

---

2. Canonical JSON

What it is. The same data can be written as text in many ways: `{"a":1,"b":2}` versus `{ "b": 2, "a": 1 }`. Those give different hashes even though the meaning is identical. Canonical JSON fixes one exact way to write data: keys sorted, no spaces, UTF-8.

Why. The log is written by one program and verified by another, possibly on a different machine. Both must produce byte-for-byte identical text, or an honest log would fail verification.

Where. `src/core/hashing.py` (`canonical_json`). **Test.** `test_canonical_json_is_order_independent`.

---

3. Hash chain

What it is. Each log entry stores the fingerprint of the entry before it (`prev_hash`) and its own fingerprint (`entry_hash`), computed over its index, timestamp, content and `prev_hash`. The first entry points at an all-zero "genesis" value.

```
entry 0                 entry 1                 entry 2
index, time, content    index, time, content    index, time, content
prev = 000...0          prev = hash(entry 0)    prev = hash(entry 1)
hash = H(entry 0)       hash = H(entry 1)       hash = H(entry 2)
```

What it protects against.

Attack	What the verifier sees
Edit an entry's content	Its stored hash no longer matches its content
Delete an entry in the middle	The next entry's prev_hash points at something missing, and indexes skip
Insert an entry	Indexes and links break
Reorder entries	Links break

What it does not protect against. A patient attacker who edits entry 3 and then recomputes the hashes of every later entry produces a chain that is internally consistent. The hash chain alone cannot tell. That is exactly why the blockchain anchor exists (section 5).

Where. `src/domain/chain.py` (`make_entry`, `append_entry`, `verify_chain`). **Tests.** `tests/test_audit.py`, `tests/test_fuzz_tamper.py`.

---

4. Merkle tree

What it is. A way to fold many fingerprints into one. Pair up the entry fingerprints and hash each pair; pair up the results and repeat until one fingerprint remains: the **Merkle root**.

```
          root
         /    \
      h(0,1)  h(2,3)
      /  \     /  \
    e0   e1   e2   e3      <- entry hashes
```

Change any single entry and the root changes. And you can prove that one entry belongs to a root by showing only the "sibling" fingerprints along its path (an **inclusion proof**), without revealing the other entries.

Design details in this project (they matter for security):

- Leaves are hashed as `sha256(0x00 || entry_hash)` and inner nodes as `sha256(0x01 || left || right)`. The different prefixes stop an attacker from passing off an inner node as a leaf.
- If a level has an odd number of nodes, the unpaired one is carried up unchanged. It is not duplicated, because duplicating the last leaf lets two different logs share one root.

Where. `src/domain/merkle.py`. **Tests.** `tests/test_audit.py` (proofs for trees of 1 to 100 entries), `tests/test_vectors.py` (checks the format against a by-hand computation).

---

5. Blockchain, Ethereum and smart contracts

Blockchain. A shared, append-only record that many computers keep copies of. New data is grouped into blocks that point at the previous block by hash. Rewriting history would mean redoing the work and convincing the network, which is impractical. For this project a blockchain is simply a place where a fingerprint, once written, cannot be quietly changed, even by the owner of the agent.

Ethereum and accounts. Ethereum is a blockchain that can run small programs. Every user or program has an **account** with an address (for example `0xf39F...`). A **transaction** is a signed request from an account, such as "call this function".

Smart contract. A program stored on the blockchain at its own address. Its code and its stored data are visible to everyone, and nobody can alter the code after deployment. Callers can only use the functions it exposes.

Gas. Every operation costs "gas", which on a real network is paid in cryptocurrency. Writing data is expensive, reading is free. This is why the project stores only one 32-byte root per batch of entries, never the log itself. Gas per anchor call is one of the project's measured results.

Local chain vs real network. This project runs on a **local development chain** (Hardhat) on the laptop: it behaves like Ethereum, but the money is fake, it is instant, and it is free. A public test network ("testnet") is an optional later step.

Honest limit. A blockchain proves the data was not changed after it was written. It cannot prove the data

was true when written.

---

6. The contract: AuditAnchor.sol

File: `contracts/AuditAnchor.sol` (Solidity 0.8.28). In plain words:

Function	Who calls	What it does
anchor(root, count)	An agent's account	Appends a batch (root, first entry index, number of entries, block time). Batches must be contiguous: the first index is the previous batch's first index plus its count. Rejects empty batches. Emits a BatchAnchored event.
batchCount(agent)	Anyone (free)	How many batches this agent has anchored.
getBatch(agent, id)	Anyone (free)	Reads one batch. Reverts if it does not exist.
verifyInclusion(agent, id, entryHash, siblings, sides)	Anyone (free)	Re-computes the Merkle path on-chain and returns true only if it ends at the stored root.

Properties by design: batches are stored per sending address, so one agent cannot overwrite or forge another's history, and nothing in the contract can edit or delete a batch once written.

Tests. `chain-tests/AuditAnchor.test.js` (18 tests).

---

7. Solidity and Hardhat

Solidity is the programming language for Ethereum smart contracts. It looks a bit like JavaScript. The compiler (`solc`) turns it into bytecode that the blockchain runs plus an **ABI**, a description of the functions so other programs can call them.

Hardhat is a development toolkit: it downloads the Solidity compiler, compiles contracts into `artifacts/`, runs a local blockchain (`npx hardhat node`), and runs tests written in JavaScript using Mocha (test runner), Chai (assertions) and ethers (talks to the chain).

Where. `hardhat.config.js`, `package.json`, `chain-tests/`. **Commands.** `npx hardhat compile`, `npx hardhat test`.

---

8. web3.py: Python talking to the chain

What it is. A Python library that sends JSON-RPC requests to a blockchain node (here the Hardhat node on `127.0.0.1`). The project uses it to deploy the contract, send `anchor` transactions, and read the stored batches.

Where. `src/app/chain_client.py`. It also starts and stops the local Hardhat node for the demo (`local_chain()`), reads the compiled contract from `artifacts/`, and decides which batches still need anchoring by asking the chain itself, so running it twice never anchors a batch twice.

Test. `tests/test_chain_integration.py`, which uses a real local chain.

---

9. LLM agents and tool calling

LLM (large language model): a program that continues text. Given a conversation it predicts a sensible reply.

Tool calling. The model can be told "you may use these functions" (a calculator, a word counter, a capital-city lookup). Instead of answering directly it can reply "call `calculator` with `12*(3+4)`". Our program runs the tool and sends the result back, and the model decides what to do next.

Agent loop. Repeat: ask the model, if it asks for a tool run the tool and append the result, if it gives a final answer stop. A step limit prevents endless loops.

What gets logged. For each loop: the task, the model's response (plus model name and token counts), the tool call, and the tool result or error, then the final answer.

Why the LLM sits behind one function signature. Every model, whether a scripted stand-in, a local model or Claude, is wrapped as `llm(messages) -> action`. The audit layer never needs to know which model produced the steps.

Where. `src/domain/agent.py`. **Tests.** `tests/test_tools.py`, `tests/test_agent_and_audit.py` .

---

10. Ollama and qwen3:8b: the local model

Ollama runs open-weight language models on your own computer and serves them over a local web API at `http://127.0.0.1:11434`. **qwen3:8b** is an 8-billion-parameter open model (about 5 GB) that supports tool calling. No account, API key or internet connection is needed to use it after the one-time download, and nothing you ask it leaves the machine.

Settings used. Temperature 0 and a fixed seed so runs are as repeatable as possible; "thinking" mode off so replies are short; one tool call at a time.

Where. `src/app/ollama_llm.py` (plain standard-library HTTP). **Tests.** `tests/test_ollama_llm.py` (fake server) and `tests/test_ollama_live.py` (the real model, skipped when Ollama is not running).

---

11. Claude through the Anthropic API (optional)

The same agent can use Claude instead. This needs an API key from Anthropic in the environment variable `ANTHROPIC_API_KEY`. The adapter (`src/app/anthropic_llm.py`) is unit-tested against a fake SDK client; it has not been run against the real service because the development setup has no key. It is kept as an option, not a dependency.

---

12. JSON Lines: where the full log lives

The complete log (the "off-chain" part) is a text file with one JSON object per line: `research/experiments/agent_logs/<name>_run.jsonl`. Plain text means it can be opened, searched and, deliberately in the demos, edited to show the verifier catching it. Each line holds the entry's index, timestamp, payload, `prev_hash` and `entry_hash`.

Where. `src/app/logstore.py`. **Tests.** `tests/test_logstore.py`.

---

13. pytest: the Python test framework

Tests are small functions starting with `test_`; `assert` states what must be true. pytest finds them, runs them, and reports failures with the values that differed. Settings are in `pytest.ini`. Tests that need a running service (Hardhat, Ollama) carry the marker `live` and skip themselves when the service is missing.

---

14. Streamlit (planned)

A Python library that turns a script into a web page. Planned for Month 4: a dashboard showing the agent's actions as a timeline, each marked verified or tampered. Not built yet.

---

15. Git

Version control: every saved state of the project is a commit. The project's history (scaffold, Hardhat, agent and anchoring, Claude client, local model) is in `git log`.

---

16. A walk through one entry

1. The agent decides to call `calculator` with `12*(3+4)`.
2. The logger writes entry 2: `{"type": "tool_call", "tool": "calculator", "args": {...}}` with its timestamp.
3. Canonical JSON turns the entry body into exact bytes; SHA-256 turns those into `entry_hash`; the entry also stores entry 1's hash as `prev_hash`.
4. After five entries the pipeline builds a Merkle tree over their hashes and sends the 32-byte root to `anchor()`. The contract records it with the block time.
5. Later the verifier reloads the log file, recomputes every hash and link (hash chain check), recomputes each batch's Merkle root and compares it with what the contract holds (anchor check).
6. Any mismatch is reported with the entry or batch where it starts.

---

17. Glossary

Term	Meaning
Agent	An LLM that can call tools in a loop to finish a task
Anchor	Write a Merkle root to the blockchain
Audit trail	A chronological record of actions that can be checked later
Batch	A group of consecutive log entries anchored together
Gas	The fee unit for blockchain operations
Genesis hash	The all-zero prev_hash of the first entry
Hash / fingerprint	A fixed-size digest of data
Inclusion proof	The sibling hashes showing one entry belongs to a Merkle root
Off-chain / on-chain	Stored in ordinary files / stored on the blockchain
Tamper-evident	Changes cannot be prevented but will be detected